

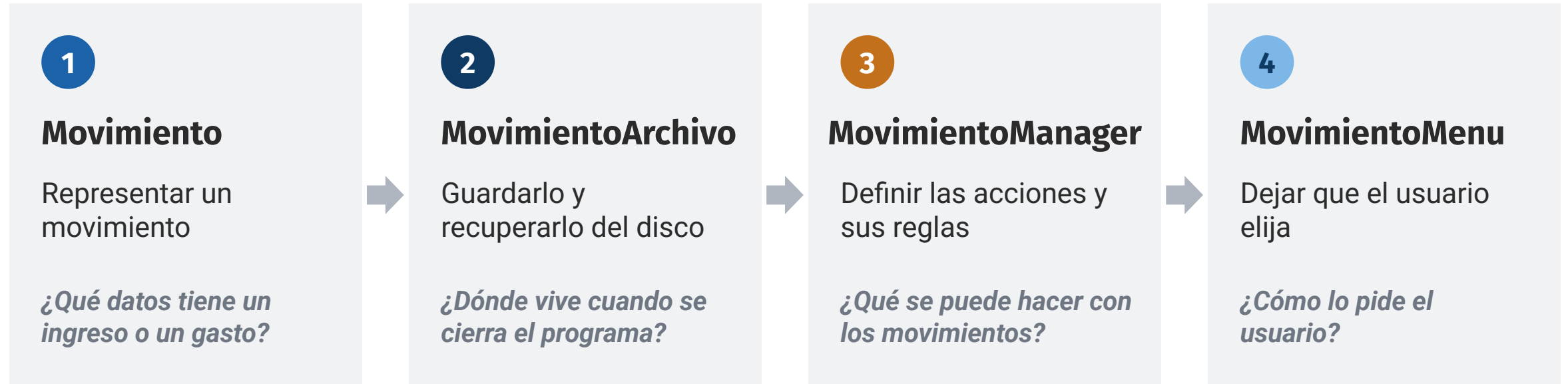
Archivos

Aplicación de control de gastos

Separar en clases y guardar en archivos

Lo vamos a representar cuatro clases

De lo más particular a lo más general: cada clase aparece cuando aparece una necesidad nueva.



El camino que vamos a recorrer

De lo más particular a lo más general: cada clase aparece cuando aparece una necesidad nueva.

1

Movimiento

Representar un movimiento

¿Qué datos tiene un ingreso o un gasto?

El camino que vamos a recorrer

De lo más particular a lo más general: cada clase aparece cuando aparece una necesidad nueva.

2

MovimientoArchivo

Guardarlo y recuperarlo del disco

¿Dónde vive un Movimiento cuando se cierra el programa? y ¿cómo recupero la información cuando la necesito?

El camino que vamos a recorrer

De lo más particular a lo más general: cada clase aparece cuando aparece una necesidad nueva.

3

MovimientoManager

Definir las acciones y sus reglas

¿Qué se puede hacer y no se puede hacer con los movimientos?

El camino que vamos a recorrer

De lo más particular a lo más general: cada clase aparece cuando aparece una necesidad nueva.

4

MovimientoMenu

Dejar que el usuario elija acciones para hacer con movimientos

¿Cómo lo pide el usuario?

Movimiento

Necesidad: representar un movimiento

¿Qué datos tiene un ingreso o un gasto y cómo se accede a ellos?

Movimiento: los datos de un movimiento

Agrupar en un solo objeto los siete datos de cada ingreso o gasto.

- IdMovimiento
- Descripción
- TipoMovimiento
- Fecha
- IdCategoria
- Importe

Lo que sabe

- Qué datos forman un movimiento
- Leerlos y modificarlos con getters y setters

Lo que no sabe

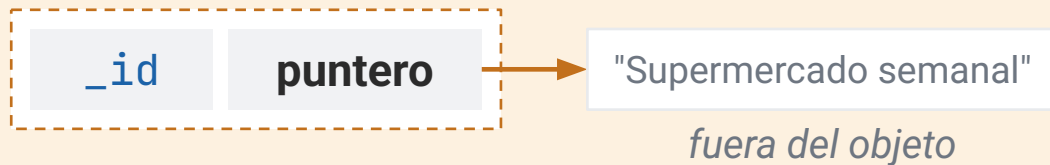
- Si está guardado en un archivo
- Si su id ya existe
- Si 'X' es un tipo válido
- Cómo se muestra en pantalla

¿Por qué char[50] y no string?

Movimiento todavía no guarda nada, pero tiene que tener una forma que otra clase pueda guardar.

Con std::string

El objeto guarda un **puntero** al texto. `fwrite` copia el puntero, no las letras.



Con char[50]

Las letras están **dentro** del objeto. `fwrite` copia todo lo necesario y todos los registros miden lo mismo.



```
std::string Movimiento::getDescripcion(){
    std::string descripcion = _descripcion;
    return descripcion;
}

void Movimiento::setDescripcion(std::string descripcion){
    strcpy(_descripcion, descripcion.c_str());
}
```

MovimientoArchivo

Necesidad: que los movimientos sobrevivan al cierre del programa

¿Cómo se guarda un movimiento en disco y cómo se recupera?

Un registro son bytes

`fwrite` copia los `sizeof(Movimiento)` bytes del objeto, tal como están en memoria.

1 registro → 84 bytes de datos

Todos miden lo mismo

Cada movimiento ocupa 84 bytes, sea cual sea el largo de su descripción.

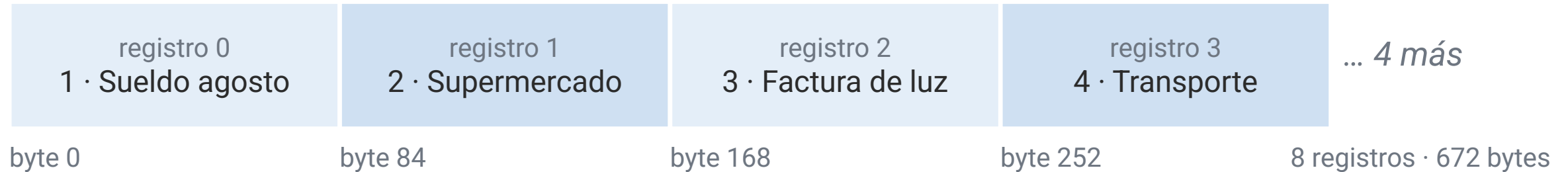
Nunca escribir 84 a mano

Se usa `sizeof(Movimiento)`: si mañana cambia el registro, el valor se actualiza solo.

El archivo: una tira de registros

movimientos.dat no tiene separadores ni nombres de campos: registros de 84 bytes, uno detrás del otro.

movimientos.dat



MovimientoArchivo

Es la única clase que abre movimientos.dat. Ninguna otra usa fopen, fread ni fwrite.

<code>bool guardar(Movimiento mov)</code>	Guarda un registro de movimiento al final del archivo. Es decir, agrega uno.
<code>Movimiento leer(int pos)</code>	Lee un registro de movimiento en la posición indicada.
<code>void leer(int cantidad, Movimiento *)</code>	Lee varios registros de movimiento y los guarda en el vector recibido
<code>int getCantidadRegistros()</code>	Devuelve la cantidad de registros del archivo
<code>int buscar(int idMovimiento)</code>	Busca un idMovimiento en el archivo y devuelve su posición si existe o -1 si no existe.

MovimientoManager

Necesidad: definir qué se puede hacer con los movimientos

MovimientoManager: las acciones del programa

Cada acción es un método. Usa a MovimientoArchivo para leer y guardar, y a Movimiento como dato.

NuevoMovimiento()	Pide los datos, aplica las reglas y guarda
ListarMovimientos()	Muestra todos en el orden de carga
ListarMovimientoXId()	Busca un id y lo muestra
ListarMovimientosXTipo()	Filtra ingresos o gastos y totaliza
MostrarResumen()	Calcula ingresos, gastos y saldo

MovimientoMenu

Necesidad: que el usuario elija qué hacer

¿Cómo se convierte una opción del teclado en una acción?

MovimientoMenu: mostrar opciones y delegar

No abre archivos ni conoce reglas: convierte un número en una llamada al manager.

```
===== CONTROL DE GASTOS E INGRESOS =====  
1. Nuevo movimiento  
2. Listar movimientos  
3. Buscar movimiento por id  
4. Listar movimientos por tipo  
5. Resumen de ingresos y gastos  
0. Salir
```